Guide démo live — Demo 2 (parcours utilisateur MaaS)

Public : client / décideur / audience technique
Durée cible : 10–12 minutes
Pré-requis : Demo 1 déjà prête, `make demo2-verify` vert, deux sessions navigateur prêtes (`admin` et `maas-demo2-user`).

Idée simple

Demo 1 a montré comment **publier** un modèle en MaaS.
Demo 2 montre comment **gouverner son usage** :

1. qui a accès
2. avec quelle limite
3. ce qui se passe quand ça casse
4. ce que voient l'admin et l'utilisateur final

La version la plus claire en live est : - **dashboard** pour la narration - **un vrai workbench utilisateur** pour prouver les appels HTTP - **un terminal admin** seulement pour restaurer exactement l'état voulu quand nécessaire

Avant d'entrer en salle

1. `make demo2-verify`
2. Vérifier que l'utilisateur `maas-demo2-user` peut se connecter
3. Avoir un workbench utilisateur prêt, ou au moins le projet `rhoai-maas-demo` visible
4. Avoir sous la main : - `secrets/demo2/maas-demo2-user.apikey` - `secrets/demo2/maas-demo2-user.ratelimit.apikey`
5. Avoir un terminal admin ouvert dans le repo

Fil rouge à dire

> « Demo 1 a publié le modèle. Demo 2 répond à la question qui intéresse la plateforme et le métier : qui peut l'appeler, avec quelles limites, et qu'est-ce que l'utilisateur voit vraiment ? »

Parcours recommandé

Étape 0 — Sans accès, l'utilisateur ne peut rien faire

À montrer seulement si tu repars volontairement d'un état "sans gouvernance".

Session `maas-demo2-user` : 1. **Gen AI studio** → **API keys** 2. Montrer : - *Request a subscription* - *No API keys* - *No subscriptions available* 3. Ouvrir quand même **Create API key** pour montrer que le sélecteur de souscription est réellement bloqué

Message :

> « Sans subscription admin, un vrai utilisateur non-admin ne peut même pas créer une clé exploitable. »

Étape 1 — L'admin montre la gouvernance

Console OpenShift, vue YAML

Montrer `MaaSSubscription` puis `MaaSAuthPolicy` dans `models-as-a-service`.

Points à verbaliser : - `MaaSSubscription` = **combien** l'utilisateur a le droit de consommer - `MaaSAuthPolicy` = **qui** a le droit d'appeler **quel** modèle - les deux sont nécessaires ensemble

Les deux objets à montrer : - `maas-demo2-subscription` - `maas-demo2-access`

Dashboard admin

Aller dans **Settings** → **MaaS governance** : - onglet **Subscriptions** - onglet **Authorization policies**

Montrer que les deux objets sont `Active`.

Si besoin pour remettre l'état correct :

```
make demo2-grant-access
```

Étape 2 — L'utilisateur crée sa clé

Session `maas-demo2-user` : 1. Retour sur **Gen AI studio** → **API keys** 2. Montrer que la subscription est maintenant visible 3. **Create API key** 4. Sélectionner `maas-demo2-subscription` 5. Montrer l'expiration max de **90 jours** 6. Créer la clé

Message :

> « La clé est liée à une subscription précise, et sa valeur n'est montrée qu'une seule fois. »

Étape 3 — Inférence réelle depuis un vrai workbench utilisateur

C'est la partie la plus pédagogique : l'utilisateur appelle le modèle depuis son propre environnement.

Dashboard utilisateur

1. **Projects** → **`rhoai-maas-demo`**
2. Ouvrir un workbench existant, ou en créer un si nécessaire
3. Dans le terminal du workbench, vérifier :

```
echo "${#DEMO2_API_KEY}"
```

Si `0`, ne pas improviser : voir `docs/demo2/05-troubleshooting.md`.

Commandes à lancer dans le workbench

```
curl -sk --http1.1 -H "Authorization: Bearer $DEMO2_API_KEY" \
  https://maas.apps.<domaine>/v1/models | python3 -m json.tool

cat > /tmp/body.json << 'JSON'
{"model": "publishers/rhoai-maas-demo/models/facebook/opt-125m", "messages": [{"role":"u
JSON

curl -sk --http1.1 -H "Authorization: Bearer $DEMO2_API_KEY" \
  -H "Content-Type: application/json" --data-binary @/tmp/body.json \
  https://maas.apps.<domaine>/v1/chat/completions -w "\nHTTP %{http_code}\n"
```

Ce qu'il faut montrer : - `GET /v1/models` → `200` - `POST /v1/chat/completions` → `200` - vraie réponse du modèle - vrai bloc `usage`

À ne pas oublier : - toujours `--http1.1` depuis un pod/workbench - jamais de JSON inline avec `-d "{\"model\"...` dans un terminal web

Étape 4 — Ce qui se passe quand la gouvernance casse

Ici, le plus simple est : - **suppression en click-ops** - **preuve par l'appel utilisateur** - **restauration sûre via manifest**

4A. Supprimer l'Authorization policy

Admin : 1. **Settings** → **MaaS governance** → **Authorization policies** 2. Supprimer `maas-demo2-access`

Utilisateur : 1. Rejouer le `POST /v1/chat/completions` 2. Attendre **`HTTP 403`**

Important : - `GET /v1/models` peut renvoyer `200` avec une liste vide - la vraie preuve dramatique est donc l'appel d'inférence, pas `/v1/models`

Restauration :

```
oc apply -f manifests/demo2/11-maas-authpolicy.yaml
```

4B. Supprimer la Subscription principale

Admin : 1. **Settings** → **MaaS governance** → **Subscriptions** 2. Supprimer `maas-demo2-subscription`

Utilisateur : 1. Rejouer le `POST /v1/chat/completions` 2. Montrer **`HTTP 403`** avec le corps indiquant `requested subscription not found`

Restauration :

```
oc apply -f manifests/demo2/10-maas-subscription.yaml
```

4C. Démontrer le rate limit

Pour cette partie, utiliser la **clé dédiée** à la petite subscription `maas-demo2-subscription-ratelimit-test`.

Dans le workbench :

```
cat > /tmp/body.json << 'JSON'
{"model": "publishers/rhoai-maas-demo/models/facebook/opt-125m", "messages": [{"role":"u
JSON

for i in $(seq 1 30); do
  curl -sk --http1.1 -H "Authorization: Bearer $RATELIMIT_KEY" \
    -H "Content-Type: application/json" --data-binary @/tmp/body.json \
    https://maas.apps.<domaine>/v1/chat/completions \
    -o /dev/null -w "call $i -> HTTP %{http_code}\n"
done
```

Ce qu'il faut montrer : - quelques `200` - puis des `429`

Message :

> « Le quota n'est pas décoratif : la plateforme finit réellement par refuser les appels. »

Étape 5 — Qui voit quoi comme usage

Ici, il faut être très honnête. C'est justement ce qui rend la démo crédible.

Côté utilisateur

Montrer : - **Observe & Monitor** → **Dashboard** → **Models** - ou, à défaut, **API keys / Subscriptions**

Ce que l'utilisateur voit bien : - le modèle - les métriques de modèle visibles dans l'onglet `Models` - la limite configurée sur sa subscription

Ce qu'il ne voit pas : - sa consommation restante - une vraie vue par clé ou par utilisateur

Côté admin

Montrer : - **Observe & Monitor** → **Dashboard** → **Models** - éventuellement l'onglet **Usage**

Message exact recommandé :

> « L'onglet Usage existe visuellement sur cette build, mais il peut rester à 0 même après du vrai trafic. En revanche, l'onglet Models fonctionne bien, et les compteurs admin de Limitador sont réels côté scripts. »

Si quelqu'un veut la preuve technique hors pitch :

```
make demo2-check-usage
```

Point de vérité à annoncer : - les métriques disponibles sont utiles - mais le suivi **par utilisateur** n'est pas fiable sur cette version

Étape 6 — Révoquer et régénérer une clé

Session `maas-demo2-user` : 1. Sur une clé active, menu ⋮ → **Revoke** 2. Taper le nom exact 3. Confirmer 4. Rejouer `GET /v1/models` ou `POST /v1/chat/completions`

Ce qu'il faut dire explicitement : - sur ce sandbox, une clé révoquée renvoie **`403`** - pas le `401` annoncé par certaines docs

Ensuite : 1. Créer une nouvelle clé 2. La remettre dans l'environnement du workbench 3. Rejouer l'appel 4. Montrer que ça repart immédiatement en `200`

Cheat sheet très court

```
1. Admin montre YAML + MaaS governance
2. User crée une clé API
3. User appelle /v1/models puis /v1/chat/completions -> 200
4. Admin casse la policy -> user prend 403
5. Admin casse la subscription -> user prend 403 "subscription not found"
6. User avec la petite clé -> boucle -> 429
7. Models tab côté user/admin
8. Revoke -> 403, nouvelle clé -> 200
```

Ce qu'il ne faut pas promettre

- Ne pas promettre un vrai suivi d'usage **par utilisateur**
- Ne pas annoncer `401` pour une clé révoquée : le comportement observé est `403`
- Ne pas utiliser `AI asset endpoints` pour la génération de clé : ce chemin est bloqué sur cette version
- Ne pas improviser dans les formulaires dashboard si tu dois restaurer vite : les manifests du repo sont plus sûrs

En secours

- état de base : `make demo2-verify`
- restauration gouvernance : `make demo2-grant-access`
- détail technique : `docs/demo2/05-troubleshooting.md`
- synthèse des constats réels : `DEMO2-RESULTS.md`